JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 **202212969** 이름: 오승재 제출일: 2026-10-28

서비스 이름 영화 리뷰장

기술 조합과 버전 React 19.3.0 / Spring Boot 3.5.16 / Windows 11, JDK 18, Node.js 26.7.0

GitHub 저장소 URL https://github.com/h1he11o/jwt-assignment

소스 코드 기준 main / d77d226

영화를 좋아하는 사람으로서 영화 리뷰 사이트를 간략하게 만들어보고 싶었다. 대상은 나처럼 영화를 좋아하는 사람들을 위해 만들었다. 사용 목적으로는 자기가 본 영화를 짧게 리뷰하고 평가를 하고 또 다른 사람들이 작성한 리뷰를 읽어 볼 수 있게 한다. 로그인이 되어있으면 영화 리뷰를 작성할 수 있고 내글을 수정, 삭제를 할 수 있다. 로그인이 되어있지 않다면 다른 사람이 작성한 리뷰만 볼 수 있다.

기술 조합 선택 이유: React는 많이 들어봤고 Spring Boot는 전 과제에서 써봤기 때문에 선택했다.
제공 템플릿 변경 범위: AuthService, api.js, main.jsx

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	BoardController.java, PostStore.java, main.jsx	로그인 없이 조회 가능
로그인과 JWT 발급	완료	AuthService.java, api.js	HS256, 만료90초, 회원가입 없음
인증과 내 정보 조회	완료	AuthService.java, api.js	Bearer 헤더로 토큰 전송
본인 글 작성·수정·삭제	완료	AuthService.java, api.js	남의 글은 버튼 숨김
만료·로그아웃·오류 표시	완료	api.js, main.jsx	새로고침하면 로그아웃
DB 데이터 유지	완료	PostStore.java, DatabaseConfig.java	SQLite사용, 서버 재시작 후에도 글 유지

구조와 변경 내역

연결구조: 브라우저(React) -> api 요청 -> Spring Boot API 서버 -> SQLite(DB)

로그인 흐름: 1. POST /api/auth/login으로 이메일과 비밀번호를 보낸다. 2. BoardController의 login()에서 계정이 맞는지 확인한다. 3. 맞으면 issueToken에서 토큰을 만든다. 4. api.js에서 토큰과 사용자를 session변수에 저장

글 수정 흐름: 1. PUT /api/posts/{id}로 수정 요청을 보내는데 Authorization: Bearer토큰을 헤더에 붙인다. 2. BoardController의 update에서 currentUserId()로 토큰을 확인 후 사용자 번호를 꺼낸다. 3. existing으로 글이있는지 확인한다. 4. assertOwner()로 내 번호와 작성자 번호를 비교한다. 5. 제목과 내용을 확인한다. 6. SQLite에 저장한다.

토큰 저장: 로그인 하면 api.js의 session변수에 저장한다.
토큰 전송: 로그인 후 요청할 때 authorizationHeaders()가 Authorization: Bearer토큰을 헤더에 붙인다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
AuthService.java / 생성자, issueToken()	비어있음	비밀키 확인, 토큰 생성	글 작성, 수정 등에 로그인이 필요한데 로그인 시 토큰이 필요하기 때문
AuthService.java / currentUserId(), assertOwner()	currentUserId()는 데모 사용자 번호 반환, assertOwner()는 비어 있음	토큰을 확인하고 사용자의 번호를 꺼내서 작성자 번호와 비교	가짜 토큰과 남이 글을 수정 하는 것을 막기 위해서
api.js/ authorizationHeaders(), request(), login(), canEdit()	authorizationHeaders()는 비어있음, login()은 에러 반환, canEdit()은 true반환 request()는 401이어도 로그인 상태 해제 안함	토큰을 만들고 토큰과 사용자를 session에 저장, 401이면 clearSession()으로 로그아웃, 로그인한 사용자 번호와 작성자 번호가 같으면 true반환	로그인 상태에서 요청, 만료되면 다시 로그인, 남의 글은 건드릴 수 없게하기 위해
main.jsx / describeError(), run()	describeError()은 없었고 run()은 오류가 나면 서버 메시지를 보여줌	describeError()에서 각 응답마다 다른 안내 문구를 만들고 401은 code에 따라 문구를 나눔 run()은 오류가 나면 describeError()로 안내를 보여주고 404면 목록을 새로 불러옴	오류가 난 이유에 대해 알려 주기 위해서

출처: 위 표의 파일과 함수 모두 AI의 초안을 검토하고 이해를 한 후 코드에 주석을 달았다.

로컬 실행 기록

운영체제와 버전: Windows 11, JDK 18 / Node.js 26.7.0

새 폴더 설치:
새 폴더에 git clone → backend-spring에서 JWT_SECRET 설정 후 .₩mvnw.cmd package → java -jar target/jwt-board-1.0.0.jar로 실행 → frontend-react에서 npm ci → npm run build → npm run preview → 4173 접속 후 Alice 로그인과 글 작성 확인. 새 폴더에서는 DB가 자동으로 새로 만들어져 글 0개로 시작했다. (docs/evidence/clone, clone_backend, clone_4173)

환경설정: PowerShell에서 $env:JWT_SECRET에 새로만든 키를 넣음
프론트 빌드: cd frontend-react → npm ci → npm run build → npm run preview
백엔드 실행: cd backend-spring → .₩mvnw.cmd spring-boot:run
SQLite 파일 경로: backend-spring/data/board.sqlite

4173번 접속 및 API 연결 확인: 서버 실행 후 http://127.0.0.1:4173에 접속해서 글 목록이 표시된것을 확인했고 터미널에서 GET /posts로 글 목록들이 나오는걸 확인했음

서버 재시작 후 데이터 유지: 서버 종료후 다시 켰을때 전에 저장된 글 들이 사라지지 않고 남아 있음

증거: docs/evidence폴더의 T01(서버 실행후 GET /posts 메소드 실행했을때 나온 글 목록들), T15-서버끄기전, T15-재시작후글목록(서버 종료후 다시 켰을때 사라지지않은 게시글들) 사진들, 4173화면 사진

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: BoardController / login() AuthService / issueToken()

코드 발췌

BoardController / login()

```
public Object login(@RequestBody Map<String, Object> body) {

   var input = fields(body, Map.of("email", 254, "password", 128));

   var user = users.verify(input.get("email"), input.get("password"));

   if (user == null) throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를
확인하세요.");

   return Map.of("accessToken", auth.issueToken(user), "expiresIn", ttl, "user", user);

 }
```

AuthService / issueToken()

```
public String issueToken(Users.PublicUser user) {          // User자바 파일의 PublicUser class,
로그인한 회원

    long now = System.currentTimeMillis();                  // 지금 시각

    return Jwts.builder()                                   // 토큰 만들기 도구

            .subject(String.valueOf(user.id()))             // sub, 토큰의 주인 (숫자에서 문자열로
바꿈)

            .issuer(issuer)                                 // iss , 발급한 곳 "webservice-class"

            .audience().add(audience).and()                 // aud, 사용처 "board-api"

            .issuedAt(new Date(now))                        // iat, 발급시간은 지금(밀리초)

            .expiration(new Date(now + ttl * 1000L))        // exp 만료시간은 지금 시간 + 90초

            .signWith(key, Jwts.SIG.HS256)                  // 비밀 키와 HS256 방식으로 서명을 만듦

            .compact();                                     // 한 줄 문자열로 만듦

  }
```

동작과 응답 코드 설명

입력 검증: fields로 이메일과 비밀번호를 확인한다. 만약 입력 형식이 다르면 400 BAD_INPUT를 보내고 토큰을 만들지 않는다.
계정 확인: users.verify로 이메일과 비밀번호가 맞는지 확인하고 만약 틀리거나 없는 계정이면 401 INVALID_CREDENTIALS를 보내고 토큰을 생성하지 않는다.
토큰 클레임: issueToken()으로 토큰을 만든다. 토큰에 sub, iss, aud, iat, exp가 들어간다.
유효 시간: exp = iat(지금시간) + 90초이다.
응답: 성공했을시 200 / 해당 없음, 실패했을시 400 / BAD_INPUT과 401 / INVALID_CREDENTIALS

3 2 브라우저 토큰 저장과 전송

파일 / 함수: api.js / session, clearSession(), authorizationHeaders(), request(), login()

코드 발췌

```
let session = { accessToken: "", user: null };
// 로그인 정보를 담는 상자, 이름은 session
export function clearSession() {
  session = { accessToken: "", user: null }; // session 상자를 처음 상태로 되돌림
}

function authorizationHeaders() {
  if (!session.accessToken) return {}; // 토큰이 없으면 빈 상자를 반환
  return { Authorization: `Bearer ${session.accessToken}` };
  // 토큰이 존재하면 Authorization 헤더를 만들어서 반환
}

request내부
headers: {
       "Content-Type": "application/json", ...authorizationHeaders(), ...options.headers,
},

if (!response.ok) { // 실패하면 (400, 401...)
    if (response.status === 401) clearSession(); // 401이면 로그아웃
    const error = new Error(
        // 오류 문구 만들기 message 있으면 쓰고 message없으면 code를 쓰고 code도 없으면 HTTP를 씀
      data.message || data.code || `HTTP ${response.status}`,
    );
    error.status = response.status; // 응답 코드 적기(코드 구분)
    error.code = data.code;         // 이유 적기
    throw error;                    // 오류 던지기
  }

login내부
session = { accessToken: data.accessToken, user: data.user };
```

동작과 응답 코드 설명

로그인 응답 처리: login()이 이메일과 비밀번호를 보내고 성공하면 받은 토큰과 사용자를 session에 저장한다. 200 / 해당 없음, 로그인 성공, (400 / BAD_INPUT, 401 / INVALID_CREDENTIALS), 로그인 실패
저장위치: session변수에 저장한다. 새로고침하면 사라진다.
Authorization 헤더: 토큰이 있으면 Authorization: Bearer토큰을 만들고, 없으면 빈값을 반환한다.
로그아웃: 로그아웃 버튼을 누르거나 401 / INVALID_TOKEN, 401 / TOKEN_EXPIRED 401 / TOKEN_REQUIRED가 오면 clearSession()으로 토큰과 사용자를 지운다. 403/ FORBIDDEN은 로그인 유지

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: AuthService / currentUserId

코드 발췌

```
String header = request.getHeader("Authorization");          // HTTP의 Authorization 헤더를 꺼내서 header에 줌
    if (header == null || !header.startsWith("Bearer ")) {       // header가 null이거나 "Bearer "로 시작하지 않을 시
      throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다."); // 401 TOKEN_REQUIRED 응답코드
    }
    String token = header.substring(7).trim();                   // header의 앞 7글자 "Bearer "를 떼어내고 토큰을 넘김
    if (token.isEmpty()) {                                       // 만약 토큰이 없다면
      throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다."); // 401 TOKEN_REQUIRED 응답코드
}
    Claims claims;                // 토큰의 내용을 담는 타입과 이름
    try {
      Jws<Claims> jws =                          // 검사를 통과한 토큰을 담을 곳
            Jwts.parser()                        // 검사 도구
                  .verifyWith(key)               // 비밀 키로 서명 확인
                  .requireIssuer(issuer)      // iss가 "webservice-class"인지 확인
                  .requireAudience(audience) // aud가 "board-api"인지 확인
                  .build()                       // 검사기 완성
                  .parseSignedClaims(token); // 토큰 검사 실행 (만료도 여기서 확인)
      if (!"HS256".equals(jws.getHeader().getAlgorithm())) {                  // 토큰 헤더의 alg가 HS256이 아니면
        throw new ApiError(401, "INVALID_TOKEN", "허용되지 않은 알고리즘입니다."); // 401 INVALID_TOKEN 응답코드
      }
      claims = jws.getPayload();                 // 검사를 통과한 토큰의 내용을 가져옴
    } catch (ExpiredJwtException e) {              // 토큰이 만료이면
      throw new ApiError(401, "TOKEN_EXPIRED", "로그인이 만료되었습니다. 다시 로그인하세요."); // 401 TOKEN_EXPIRED 응답코드
    } catch (JwtException | IllegalArgumentException e) { // 서명 위조, iss/aud의 값이 다르거나 토큰의 형식이 이상하면
      throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");   // 401 INVALID_TOKEN 응답코드
    }
String sub = claims.getSubject();                                             // 토큰의 sub를 꺼냄
    if (claims.getExpiration() == null || sub == null || !sub.matches("[1-9][0-9]{0,15}")) {
      // exp가 없거나, sub가 없거나, 첫자리는 1부터 9까지의 수이고 그 뒤로 0에서 9까지의 숫자를 가지고 최소 0부터 최대 15개의 숫자가 아닌가
      throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다."); // 401 INVALID_TOKEN 응답코드
```

```
    }
    long userId = Long.parseLong(sub);                    // 문자열 sub를 정수로 바꿈
    if (users.find(userId) == null) {                     // 회원 명단에 번호가 없으면
      throw new ApiError(401, "INVALID_TOKEN", "사용자를 확인할 수 없습니다."); // 401 INVALID_TOKEN
응답코드
    }
    return userId; // 유저 번호를 반환 (모든 검사를 통과함)
```

동작과 응답 코드 설명

서명: verifyWith(key)로 서버 키로 만든 서명을 확인한다. 서명이 안맞으면 401 / INVALID_TOKEN
알고리즘: 토큰 헤더의 alg가 HS256이 아니면 401 / INVALID_TOKEN을 보낸다.
만료: exp가 지나면 401 / TOKEN_EXPIRED을 보낸다.
필수 클레임: iss가 webservice-class, aud가 board-api인지 확인한다. iss, aud의 값이 다르거나 토큰의 형식이 이상하면 401 / INVALID_TOKEN을 보낸다.
사용자 ID 전달: sub의 사용자 번호가 실제 계정인지 users.find()로 확인한다. 없으면 401 / INVALID_TOKEN를 보냄

모든 검사를 통과하면 유저 번호를 반환한다. 이후 API마다 200, 201,204로 응답하고 모든 401에는 WWW-Authenticate:Bearer헤더가 붙는다.

3 4 작성자 권한 검사

파일 / 함수: AuthService / assertOwner, BoardController / create(), update(), delete()

코드 발췌

AuthService

```
public void assertOwner(long userId, long authorId) {    // 요청한 유저 번호, 글쓴 유저 번호
  if (userId != authorId) {                    // 요청한 유저의 번호와 글쓴 유저의 번호가 다르다면
    throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다."); // 403 FORBIDDEN
응답코드
}
```

BoardController

```
@PostMapping("/posts")
  public ResponseEntity<?> create(@RequestBody Map<String, Object> body, HttpServletRequest req) {
    long user = auth.currentUserId(req);
    var input = fields(body, Map.of("title", 100, "content", 5000));
    var p = store.create(input.get("title"), input.get("content"), user);
    return ResponseEntity.created(URI.create("/api/posts/" + p.id())).body(p);
  }
  @PutMapping("/posts/{id}")
  public Object update(
      @PathVariable String id, @RequestBody Map<String, Object> body, HttpServletRequest req) {
    long user = auth.currentUserId(req), pid = postId(id);
    var p = existing(pid);
    auth.assertOwner(user, p.authorId());
    var input = fields(body, Map.of("title", 100, "content", 5000));
    return store.update(pid, input.get("title"), input.get("content"));
  }
  @DeleteMapping("/posts/{id}")
  public ResponseEntity<?> delete(@PathVariable String id, HttpServletRequest req) {
    long user = auth.currentUserId(req), pid = postId(id);
    var p = existing(pid);
    auth.assertOwner(user, p.authorId());
    store.delete(pid);
    return ResponseEntity.noContent().build();
```

}

동작과 응답 코드 설명

누구와 누구를 비교하는지: update()와 delete()에서 assertOwner()에서 유저번호와 작성자 번호를 비교한다.
번호가 다르면 403 / FORBIDDEN을 보낸다. 같으면 수정 200 / 해당 없음, 삭제 204 / 본문 없음을 보냄
401과 403의 차이: 401은 누군지 모를때(토큰 없음, 변조, 만료), 403은 누군지 알지만 권한이 없을때다.
작성자 위조 방지: create()에서 currentUserId()로 번호를 꺼내서 저장한다. 요청에 authorId나 userId를 넣으면 fields()가 400 / BAD_INPUT로 거부한다. 성공하면201 / 해당없음

웹서비스설계 04

테스트 결과

실행 환경: Windows 11 / JDK 18, Node.js 26.7.0 / Chrome / 백엔드 .₩mvnw.cmd spring-boot:run, 프론트 npm ci -> npm run build -> npm run preview

로컬 배포 확인:
새 폴더에 git clone → backend-spring에서 JWT_SECRET 설정 후 .₩mvnw.cmd package → java -jar target/jwt-board-1.0.0.jar로 실행 → frontend-react에서 npm ci → npm run build → npm run preview → 4173 접속 후 Alice 로그인과 글 작성 확인. 새 폴더에서는 DB가 자동으로 새로 만들어져 글 0개로 시작했다. (docs/evidence/clone, clone_backend, clone_4173)

테스트 일시 및 코드 버전: 2026-10-27 ~ 2026-10-28 / main d77d226

토큰 TTL: 일반 테스트 90초 / 만료 테스트 값 3초

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T01	토큰 없이 목록 및 존재하는 글 상세 조회	200, 정상 공개 조회	전부 200 / 해당 없음	PASS	docs/evidence/T01
T02	Alice 계정으로 로그인	200, JWT와 공개 사용자 정보, 화면 로그인 표시	200 / 해당없음	PASS	docs/evidence/T02
T03	잘못된 비밀번호 및 없는 이메일로 각각 로그인	401 INVALID_CREDENTIALS, 토큰 미발급	전부 401 / INVALID_CREDENTIALS	PASS	docs/evidence/T03
T04	토큰 없이 /me와 글 등록 요청	각각 401 TOKEN_REQUIRED	전부 401 / TOKEN_REQUIRED	PASS	docs/evidence/T04
T05	Alice 토큰으로 글 작성 후 /me 조회	201, authorId 1, /me는 Alice	201 / 해당 없음(8번 게시글 생성) /me 200 / 해당 없음	PASS	docs/evidence/T05

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T06	Alice 토큰으로 본인 글 수정·삭제	200 및 204, 결과가 DB에 반영	(7번 게시글 삭제) 200 / 해당 없음, 204 / 본문 없음, 404 / "code":"NOT_FOUND","message":"게시글이 없습니다."	PASS	docs/evidence/T06
T07	Alice 글을 Bob의 정상 토큰으로 수정	403 FORBIDDEN, 원문 유지	403 / FORBIDDEN, 원문 유지	PASS	docs/evidence/T07
T08	Alice 글을 Bob의 정상 토큰으로 삭제	403 FORBIDDEN, 글 유지	403 / FORBIDDEN, 글 유지	PASS	docs/evidence/T08
T09	유효 토큰의 서명 부분 첫 글자를 다른 문자로 바꿔 /me 및 변경 요청	401 INVALID_TOKEN, 데이터 변경 없음	전부 401 / INVALID_TOKEN, 재조회 200 / 해당없음	PASS	docs/evidence/T09-1, docs/evidence/T09-2
T10	짧은 TTL로 발급하고 실제 만료 뒤 /me 및 변경 요청	401 TOKEN_EXPIRED, 화면 로그인 해제와 안내	전부 401 / TOKEN_EXPIRED	PASS	docs/evidence/T10-1, docs/evidence/T10-2, docs/evidence/T10-3
T11	로그인 후 로그아웃, 다음 /me 요청	헤더에서 토큰이 빠지고 401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED, Request Headers에 Authorization 없음	PASS	docs/evidence/T11-1, docs/evidence/T11-2, docs/evidence/T11-3, docs/evidence/T11-4
T12	Bob 로그인 상태로 글 등록 요청에 authorId:1 또는 userId:1 추가	400 BAD_INPUT, 위조 글 미생성	전부 400 / BAD_INPUT, 위조 글 미생성	PASS	docs/evidence/T12
T13	본인 토큰으로 빈 제목·공백 본문·길이 초과 입력을 각각 전송	400 BAD_INPUT, DB 변경 없음	전부 400 / BAD_INPUT, 조회 결과 변경 없음	PASS	docs/evidence/T13
T14	없는 양의 정수 ID의 조회·수정·삭제 및 잘못된 ID 조회	없는 글 404, 잘못된 ID 400; 수정·삭제에는 정상 토큰 사용	양의 정수 조회,수정,삭제 404 / NOT_FOUND 잘못된 ID 조회 400 / BAD_INPUT	PASS	docs/evidence/T14

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T15	글을 만든 뒤 서버 종료·재실행	기존 글과 작성자·내용 유지	재시작 전후 둘다 200 / 이상없음, 기존 글과 작성자, 내용 모두 동일	PASS	docs/evidence/T15-서버끄기전, docs/evidence/T15-서버끔, docs/evidence/T15-서버재시작, docs/evidence/T15-재시작후글목록
T16	로그인 후 페이지 새로고침	공개 글 유지, 메모리 로그인 상태 해제	목록 유지, 로그아웃 상태	PASS	docs/evidence/T16-login, docs/evidence/T16-F5
T17	토큰 없는 수정·삭제 요청과 Bob 계정의 정상 작성·본인 수정·삭제	무토큰은 401, Bob 본인 작업은 성공	무토큰 401 / TOKEN_REQUIRED Bob 201, 200, 204 / 해당없음 (9번 게시글)	PASS	docs/evidence/T17-1, docs/evidence/T17-2

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP와 code	증거 위치
정상 조회·로그인·수정	**200**	조회, 로그인, 수정 200 / 해당 없음	docs/evidence/T01, docs/evidence/T02, docs/evidence/T06
새 글 등록	**201 및 Location**	등록 201 / 해당 없음 Location: /api/posts/8	docs/evidence/T05
본인 글 삭제	**204 및 빈 본문**	204 / 본문 없음 (7번 게시글 삭제)	docs/evidence/T06
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	입력오류, 추가 작성자 필드 400 / BAD_INPUT	docs/evidence/T12, docs/evidence/T13
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / INVALID_CREDENTIALS	docs/evidence/T03
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	토큰없음, 헤더형식 오류 401 / TOKEN_REQUIRED	docs/evidence/T04, docs/evidence/extra
변조·클레임 오류	**401 INVALID_TOKEN**	서명을 바꾼 토큰으로 /me, 등록, 수정, 삭제 전부 401 / INVALID_TOKEN	docs/evidence/T09-1, docs/evidence/T09-2
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED	docs/evidence/T10-1, docs/evidence/T10-2
다른 작성자의 변경	**403 FORBIDDEN**	다른 계정으로 수정, 삭제 403 / FORBIDDEN	docs/evidence/T07, docs/evidence/T08
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND	docs/evidence/T14

보호 API 401의 WWW-Authenticate: Bearer 확인 결과와 증거: 401이 나왔을때 응답 헤더에 WWW-Authenticate: Bearer가 나와있는걸 확인했다.
(증거: docs/evidence/T04, T09-2, T10-2, extra)

401과 403에서 화면이 각각 어떻게 달라지는지 설명: 401이 나오면 api.js에서 clearSession()으로 로그아웃 상태로 바뀐다. (증거: docs/evidence/T10-3)
403은 clearSession()을 하지 않으므로 로그인이 그대로 유지된다. 403은 API로 요청을 확인했다. (증거: docs/evidence/T07, T08)

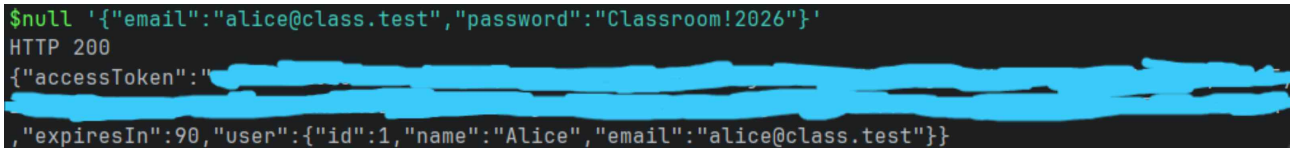
실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과: docs/evidence/T07(수정), T08(삭제), T09-2(변조 토큰), T13(입력 오류)에서 실패한 뒤 GET /posts/8로 다시 조회했을 때 데이터가 변하지 않았다.

토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404의 실제 결과: 토큰 없이 없는 글을 수정했을때(docs/evidence/extra) 응답은 401, 코드는 TOKEN_REQUIRED가 나왔고 정상 토큰으로 없는 글을 수정하면(docs/evidence/T14) 응답은 404, 코드는 NOT_FOUND가 나온다.

테스트 증거

로그인과 토큰 전송

Alice로 로그인하면 200과 토큰을 받는다. 그 토큰을 Authorization: Bearer헤더에 붙여서 글을 쓰면 201이 나온다. /me 요청도 200으로 나온다.
(증거: docs/evidence/T02, T05)




```
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> $a = Login "alice@class.test"
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call POST /posts "Bearer $a" '{"title":"1917","content":"롱테이크가 길어질수록 긴장감은 몇배로. 별점 5/5"}'
HTTP 201
Location: /api/posts/8
{"id":8,"title":"1917","content":"롱테이크가 길어질수록 긴장감은 몇배로. 별점 5/5","authorId":1,"authorName":"Alice","createdAt":"2026-10-27T11:22:24.2731664002","updatedAt":"2026-10-27T11:22:24.2731664002"}
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call GET /me "Bearer $a"
HTTP 200
{"id":1,"name":"Alice","email":"alice@class.test"}
```

다른 작성자의 수정과 삭제

Bob으로 Alice의 8번 글을 수정, 삭제 하려고 할 때 403 / FORBIDDEN이 나온다.
(증거: docs/evidence/T07, T08)

```
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> $b = Login "bob@class.
test"
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call PUT /posts/8 "Bea
rer $b" '{"title":"1917","content":"Bob이 고침"}'
HTTP 403
{"code":"FORBIDDEN","message":"본인이 작성한 글만 수정·삭제할 수 있습니다."}
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call GET /posts/8
HTTP 200
{"id":8,"title":"1917","content":"롱테이크가 길어질수록 긴장감은 몇배로. 별점 5/5","authorId":1,"auth
orName":"Alice","createdAt":"2026-10-27T11:22:24.273166400Z","updatedAt":"2026-10-27T11:22:24.2731664
00Z"}
```

```
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> $b = Login "bob@class.
test"
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call DELETE /posts/8 "
Bearer $b"
HTTP 403
{"code":"FORBIDDEN","message":"본인이 작성한 글만 수정·삭제할 수 있습니다."}
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call GET /posts/8
HTTP 200
{"id":8,"title":"1917","content":"롱테이크가 길어질수록 긴장감은 몇배로. 별점 5/5","authorId":1,"auth
orName":"Alice","createdAt":"2026-10-27T11:22:24.273166400Z","updatedAt":"2026-10-27T11:22:24.2731664
00Z"}
```

토큰 변조와 만료

서명을 바꾼 토큰은 401 INVALID_TOKEN, 만료된 토큰은 401 TOKEN_EXPIRED가 나온다. 만료 후 화면은 로그아웃 상태로 바뀐다.

(증거: docs/evidence/T09-2, T10-2, T10-3)

```
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call GET /me "Bearer $
bad"
HTTP 401
WWW-Authenticate: Bearer
{"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}

PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call POST /posts "Bear
er $bad" '{"title":"x","content":"x"}'
HTTP 401
WWW-Authenticate: Bearer
{"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}

PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call PUT /posts/8 "Bea
rer $bad" '{"title":"x","content":"x"}'
HTTP 401
WWW-Authenticate: Bearer
{"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call DELETE /posts/8 "
Bearer $bad"
HTTP 401
WWW-Authenticate: Bearer
{"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call GET /posts/8
HTTP 200
{"id":8,"title":"1917","content":"롱테이크가 길어질수록 긴장감은 몇배로. 별점 5/5","authorId":1,"auth
orName":"Alice","createdAt":"2026-10-27T11:22:24.273166400Z","updatedAt":"2026-10-27T11:22:24.2731664
00Z"}
```

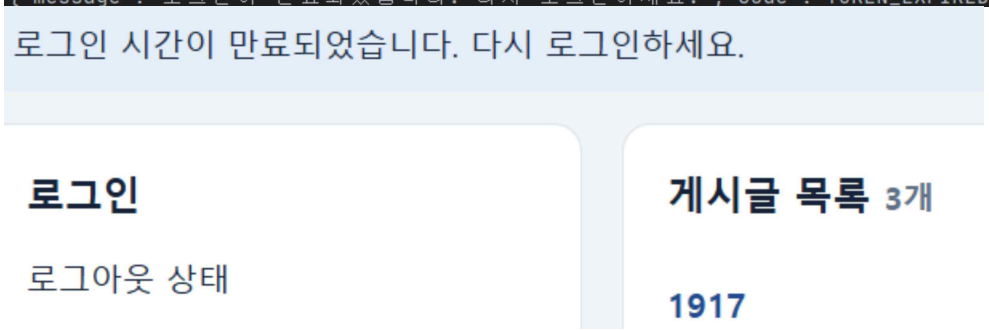
```
PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> $a = Login "alice@clas
s.test"

PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Start-Sleep 5

PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call GET /me "Bearer $
a"
HTTP 401
WWW-Authenticate: Bearer
{"message":"로그인이 만료되었습니다. 다시 로그인하세요.","code":"TOKEN_EXPIRED"}

PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call PUT /posts/8 "Bea
rer $a" '{"title":"x","content":"x"}'
HTTP 401
WWW-Authenticate: Bearer
{"message":"로그인이 만료되었습니다. 다시 로그인하세요.","code":"TOKEN_EXPIRED"}

PS C:\Users\u6862\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment> Call DELETE /posts/8 "
Bearer $a"
HTTP 401
WWW-Authenticate: Bearer
{"message":"로그인이 만료되었습니다. 다시 로그인하세요.","code":"TOKEN_EXPIRED"}
```



자동 검사와 로컬 배포 증거

자동 검사 실행 명령: py tools/check_api.py --mode final 실행 결과 모든 항목을 PASS했고 TOKEN_TTL = 3으로 재시작 한 후 py tools/check_api.py --mode final --expiry 실행 결과 모두 PASS했다.
자동 검사 실행 의미: 검사 도구가 서버에 직접 요청을 보내 로그인, 토큰 없음, 변조, 만료, 다른 사용자의 수정, 삭제, 작성자 위조, 입력 오류, 없는 글의 HTTP상태와 code를 확인한다. 모두 PASS했으므로 서버가 API 명세대로 응답한다. 이 도구는 화면, 로그아웃, 새로고침, 서버 재시작은 확인하지 않으므로 T01~T17에서 직접 확인했다.
(증거: docs/evidence/autocheck, autoexpiry)

```
PASS no token me
PASS no token create
PASS login alice@class.test
PASS login bob@class.test
PASS Alice me
PASS Bob me
PASS create post
PASS public detail
PASS no token PUT
PASS Bob denied PUT
PASS failed request leaves data unchanged
PASS no token DELETE
PASS Bob denied DELETE
PASS failed request leaves data unchanged
PASS reject authorId
PASS reject userId
PASS input validation
PASS input validation
PASS input validation
PASS input validation
PASS tampered GET /me
PASS tampered POST /posts
PASS tampered PUT /posts/10
PASS tampered DELETE /posts/10
PASS failed request leaves data unchanged
PASS owner update
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 40 checks passed (final). UI and restart tests must be performed separately.
```

```
PASS expired PUT /posts/12
PASS expired DELETE /posts/12
PASS failed request leaves data unchanged
PASS login alice@class.test
PASS login bob@class.test
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 48 checks passed (final). UI and restart tests must be performed separately.
```

빌드 성공 로그: npm run build결과 built in 298ms가 나왔다. (증거: docs/evidence/build)
4173번 API 연결: 브라우저로 http://127.0.0.1:4173에 접속했고 글 목록이 표시되었다.
(증거: docs/evidence/4173화면, T01)
재시작 유지: 서버를 끄고 다시 켠 뒤에도 글이 그대로 남아 있었다.
(증거: docs/evidence/T15-서버끄기전, T15-재시작후글목록)

웹서비스설계 05

AI 활용과 참고 자료

AI 사용 여부: 사용 (Claude Opus5.5)

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
클로드 Opus 5.5	학습(JWT개념 학습, 코드 구조 설명, 주석 검토)	AuthService, api.js, main.jsx	참고
클로드 Opus 5.5	생성(TODO코드 초안, describeError 코드 초안)	AuthService, api.js, main.jsx	그대로 반영
클로드 Opus 5.5	디버깅(BUILD FAILURE)	서버 실행 환경, 보고서 6절 5번	참고
클로드 Opus 5.5	테스트(T01~17)	보고서 4절	그대로 반영
클로드 Opus 5.5	문장정리(보고서 초안 정리)	보고서 1 ~ 6절	수정 후 반영
클로드 Opus 5.5	README 작성	README.md	그대로 반영

대표 사례

대표사례1

도구 / 날짜: 클로드 Opus 5.5 / 2026-10-27

요청한 내용: AuthService backend spring의 TODO BE 01 ~ 03을 구현해달라 하고, 문법을 몰라서 한 줄씩 설명해 달라고 했다.

AI가 제안한 내용: 비밀 키 형식 검사, HS256토큰 발급, 토큰 검증, assertOwner 작성자 비교 코드

실제 반영 위치: backend-spring의 AuthService / 생성자, issueToken(), currentUserId(), assertOwner()

내가 확인하고 수정한 내용: 코드는 그대로 반영했다. AI의 코드 설명을 보면서 한 줄씩 직접 주석을 달았다.

검증 방법과 결과: docs/evidence T02, 03(로그인 200과 401), T04, 09, 10(401 / TOKEN_REQUIRED, INVALID_TOKEN, TOKEN_EXPIRED), T07, 08(403 FORBIDDEN), 자동 검사 모두 PASS

대표사례2

도구 / 날짜: 클로드 Opus 5.5 / 2026-10-27

요청한 내용: main.jsx에 어디를 수정해야 하는지 물어보고 필요한 부분을 알려달라 했다. 그리고 만들어진 코드의 설명을 부탁했다.

AI가 제안한 내용: describeError()함수, run()의 404처리 코드 제안, 401, 403, 404안내, 비로그인 글 작성 안내, 배너 교체

실제 반영 위치: frontend-react의 main.jsx / describeError(), run(), 배너와 안내

내가 확인하고 수정한 내용: 코드와 배너 문구는 그대로 반영했다. AI의 코드 설명을 보면서 새로 만든 함수와 추가한 코드에 한 줄씩 주석을 달았다. 화면 제목은 직접 작성했다.

검증 방법과 결과: docs/evidence 4173화면(배너수정), T10-3(describeError()의 만료 안내 문구), T07, 08, 14(서버 응답으로 403, 404안내 확인)

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
과제 제공 자료 01_과제공지, 02_API명세, 03_체크리스트	AuthService, api.js, main.jsx	구현 요구사항, HTTP 상태와 code, JWT 설정값
과제 제공 자료 04_보고서양식, 05_테스트시나리오	보고서전체, docs/evidence	보고서 구성, T01 ~ T17 절차와 기대 결과
과제 제공 자료 06_제출용_README, 07_로컬실행안내	README.md, 보고서 2절, 4절	설치, 빌드, 실행 명령, 제출 방법
과제 제공 시작 템플릿(frontend-react, backend-spring)	전체	제공 코드 유지, TODO 부분만 구현
과제 제공 검사 도구 tools/check_api.py	보고서 4절 자동 검사	자동 검사 실행(docs/evidence/autocheck, autoexpiry)

이해도 확인과 회고

1. 요청 본문의 authorId를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문의 authorId를 믿으면 다른 유저가 authorId : 1을 넣어서 authorId가 1인 사람이 글을 쓴 것처럼 만들 수 있다. 내 코드에서는 create()에서 currentUserId()로 토큰에서 꺼낸 번호로 저장하고 fields()가 title, content외의 필드를 400 BAD_INPUT으로 거부한다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼은 화면에서만 숨겨지므로 PowerShell이나 curl로 서버에 직접 요청을 보낼 수 있다. assertOwner()가 토큰의 유저 번호와 글의 작성자 번호를 비교해서 다르면 403 / FORBIDDEN을 보낸다.

3. JWT decode와 verify의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode는 토큰 내용을 읽기만 하는 것이라 내용을 바꿔도 알 수 없다. verify는 비밀 키로 서명이 맞는지 확인해서 위조된 토큰을 막는다. 내 코드에서는 currentUserId()에서 verifyWith(key)로 서명을 확인하고 iss, aud, exp, sub가 맞는지 확인하고 alg가 HS256인지, sub의 사용자가 실제로 있는지도 확인한다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 clearSession()으로 브라우저 메모리의 토큰과 사용자를 지운다. 토큰 만료는 만료시간이 지나서 서버가 401 TOKEN_EXPIRED를 보내고 프론트가 clearSession()으로 로그아웃시킨다. 새로고침은 토큰이 session 변수에만 있어서 사라지고 로그아웃 상태가 된다. 이미 복사된 토큰은 로그아웃을 해도 서버에서 무효되지 않으므로 만료시간이 지나기 전까지는 그 토큰으로 계속 요청할 수 있다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

AuthService를 구현하고 서버를 실행했는데 BUILD FAILURE가 났다. 이유는 JWT_SECRET이 설정되지 않아서 서버가 시작을 거부한 것이었다. 예시 키를 넣었는데 예시 키는 거부하도록 설정되어있어서 실패했다. node 명령으로 새 키를 만들어 PowerShell에서 $env:JWT_SECRET에 넣었고 서버는 정상 실행되었다. 새 PowerShell창에서 다시 서버를 킬때 같은 오류가 났는데 환경변수는 새 창마다 다시 넣어야 한다는 것을 알게되었고 키를 넣고 서버를 킨 후 GET /api/posts가 200 / 해당 없음으로 나온것을 확인했다.

현재 남은 문제와 다음에 개선할 점:
로그아웃을 해도 이미 복사된 토큰은 만료 전까지 유효하다. 또 백엔드가 꺼져 있으면 화면에 영어 오류 문구가 그대로 보이는데, 다음에는 연결 실패 안내 문구를 따로 보여주고 싶다.

제출 확인: [V] 실제 코드와 설명 일치 [V] 테스트 직접 수행 [V] AI 활용 및 출처 기록 [V] 비밀 키와 토큰 제거

GitHub 확인: [V] 루트 README 작성 [V] 보고서 PDF와 증거 업로드 [V] README 문서 링크 확인 [V] 교수자 접근 가능 [V] 저장소 URL과 커밋 SHA 제출